Hadley McCormack
AIPI 520: Modeling Process and Algorithms
November 4, 2026

Kaggle Competition Write Up

For this competition, our task was to build a model that predicts which previous patrons of the Philharmonia Baroque Orchestra will purchase a subscription to the 2014-15 concert season (the target is whether the patron purchased a 2014-15 season subscription (0/1 label)). We were given 6,941 labeled patrons for training and 2,975 unlabeled patrons for testing, which were split into a public part and a private part. Predictions for each patron are submitted as probabilities and scored using AUROC. AUROC is a good metric for this because only 4.8% of patrons in the training data bought a 2014-15 subscription, so there is heavy class imbalance. If we used a metric like accuracy, a model could achieve a 95.2% accuracy score just by predicting the common class for everyone, which is a useless model.

The model I went with as my final choice is a rank-average ensemble of gradient boosting, a random forest, and a logistic regression that makes predictions using subscription, ticket, and donation history features.

Data and EDA

To start, we were given a subset of patrons labeled with the target 0/1 indicating whether they purchased a 2014-15 season subscription to be used for training (train.csv) and an unlabeled subset of patrons for us to generate predictions for (test.csv). We also were given six other datafiles: account.csv (contains one row per patron (19,833) with zip code, city, and donation history), tickets_all.csv (2,808 single-ticket orders (2,049 patrons) from the 2010-11 season to 2013-14), subscriptions.csv (28,627 past subscription rows (6,267 patrons) from the 1993-94 season to 2013-14), concerts.csv (contains information on the programs from the 2010-11 to 2013-14 seasons), concerts_2014-15.csv (contains information on the six planned programs for the 2014-15 season), and zipcodes.csv (location and demographic information per zip code, including population, tax returns, and total wages). The three models in the final ensemble use the same 17 features from a patron's history before 2014-15: 10 subscription history features, 3 ticket history features, and 4 donation history features. concerts.csv, concerts_2014-15.csv, and zipcodes.csv were not used in the final model.

EDA showed the following important information about the data. First, it revealed a heavy class imbalance: only 4.8% of patrons bought. Second, train and test look alike: about 32% of patrons in each have subscription history and about 10% have ticket history, meaning a score on train should carry over to test. Third, last season's subscription is the strongest signal. 345 patrons subscribed in 2013-14, and 82% of them bought again; among the other 6,596 patrons, only 0.8% did. Fourth, it showed that more seasons subscribed means a higher buy rate. It rises from 3% at one season to 56% at ten or more. There are 323 patrons who have subscribed for ten or more seasons, suggesting long-time subscribers. Fifth, donors are much more likely to buy, with 14% of donors buying in comparison to only 1.2% of non-donors. Sixth, donation amounts are very skewed: the median is 0, the 99th percentile is about $17,800, and the maximum is about $2.3 million. Lastly, in the account file, shipping.zip.code, shipping.city,

and relationship are almost entirely missing (97-99%) and first.donated is 72% missing. Billing zip is also 15% missing and has mixed formats.

The history files only go up to 2013-14, so no feature contains the 2014-15 outcome. This is important because there is no data leakage, meaning a feature secretly contains the answer or the model is able to "peek" at the test set. If there is data leakage, the metrics may be overoptimistic and thus not an honest representation of model performance.

Data Pipeline

After EDA, the next step was to use this information for preprocessing (data cleaning) and feature engineering. The models need one row per patron, so from the raw history tables in the given data files, all rows with the same patron ID were grouped and collapsed into single summary numbers like counts, averages, sums, or earliest and latest year, which became the input features to the models.

In terms of data cleaning, I converted season text like "2013-2014" into just a start year, so in this example just 2013. Patrons with no subscription or ticket history had missing values after joining, which were filled with 0 to mean no history, except for years since last subscription, where missing values were filled with 30 to mean never or a very long time ago. The account columns that were found in EDA to be almost entirely empty (shipping.zip.code, shipping.city, and relationship) were dropped, and billing zip was also not used in the final model. I also engineered features like years_since_last_sub (2014 minus the last subscription year) and binary flags for whether a patron had a subscription in certain years and whether or not they were a donor (sub_in_2013, sub_in_2012, and sub_in_2011 and has_donated). I chose to engineer these features since someone who subscribed last season is likely to renew whereas someone who stopped subscribing in a previous year likely won't. These features allow the model to see this. The has_donated flag is included since donors likely have a deeper commitment to the orchestra, so they should be more likely to buy. EDA supported both of these ideas.

Scaling was only used for logistic regression since the tree models only care about the ordering of values and thus don't need scaling. Here, I applied log1p, log of 1 plus the value, and then standardized each feature using only the training data to prevent data leakage.

Donation amount was the only feature where outliers were a concern, and I didn't remove or cap anything. The log transform helped deal with the outliers for the logistic regression, and for the trees, it doesn't matter because they use order rather than magnitude, so they are more robust to outliers.

The 17 features were hand-picked, and the findings from EDA supported them. No feature selection was used. Extra feature groups were tested (in cv.py), but none of these were used in the final model.

Validation Approach

Since Kaggle's test answers are hidden, I had to make my own unseen patrons out of the given training data by hiding some of the answers and testing on them. I had to do this because you can't judge a model from how it does on the data it learned from. Initially, I used one split (testing on one hidden group) for validation, but the score could end up good or bad just because of luck since only about 5% of patrons buy and one hidden group therefore has

very few buyers. I saw this with that single split, where I got a score of 0.975 when the true level was about 0.963. I shifted to using cross-validation, which split the patrons into 5 groups, hid one group at a time, trained on the other four, and scored on the hidden one. With this, every patron gets tested once. This was repeated three times with different random splits, and I averaged the fifteen scores, thus giving a more robust measure of the model's performance since using many hidden groups averages out the luck. I used this to compare the models fairly since with cross-validation, I am picking the model that scores better across many splits rather than the one that got lucky. Since there is class imbalance (only 4.8% are buyers), I used stratified folds so that every fold has the same share of buyers. Every candidate was scored on the same folds. I also set an adoption rule at the start: a model needed a gain of at least 0.002 and wins in at least 11 out of the 15 folds to be chosen as the better model. I intended to use the public leaderboard scores mostly as a sanity check.

Modeling and Decisions

I started with gradient boosting on the 17 patron history features as a baseline because it can capture interactions and needs very little preprocessing. It also was a good first choice because it can handle tabular data with mixed scales. I tried eight different settings and ultimately settled on 100 trees, a depth of 3, and a learning rate of 0.05. The cross-validation AUROC for this was 0.9633, and the public test AUROC was 0.97665.

The next step was to see if more information helped, so I added more detail from the given subscription, ticket, and donation data plus zip code information. None of these additions clearly helped, so I stuck with the original 17 features.

Next, I decided to use ensembles to see whether combining models helped. The idea came from our lecture on ensembles in class, where we talked about how models making different mistakes partly cancel each other out, which also makes ensembling a good option for avoiding overfitting. The three candidate models for the ensemble were gradient boosting, logistic regression, and random forest. To combine them, I used rank averaging, meaning that instead of averaging the predictions each model gives for a patron, I converted each model's predictions to ranks and averaged those. I did this because each model in the ensemble produces output on a different scale, so the numbers don't mean the same thing across models. Averaging them directly may lead to one model counting for more than the others just because of the differing scales. With ranks instead, every model's output is on the same 0 to 1 scale, so when the predictions are averaged, each model gets an equal say. Averaging ranks is safe here because AUROC is the metric. AUROC only checks whether patrons who bought are ordered above patrons who didn't, never using the actual values, so converting to ranks doesn't lose anything that the score actually looks at.

Gradient boosting builds trees one after another, where each new tree is trained on the errors of the combined model so far, thus improving upon the mistakes of the earlier trees. Random forest grows many decision trees, each trained on a different bootstrapped sample of the training data and averages their predictions. Logistic regression feeds a weighted sum of features into the sigmoid function, giving a probability between 0 and 1 that the patron buys. Cross-validation showed that these three models averaged together beat the baseline of gradient boosting alone (0.9682 vs. 0.9633) in 13 out of 15 folds. An ensemble of only random forest and gradient boosting did worse than the baseline (0.9583), but adding the logistic

regression made the ensemble work. Logistic regression alone also achieved a cross-validation AUROC of 0.9663, which goes back to the point that a few simple patterns carry most of the signal. Lastly, I tested a variant with zip features that looked slightly better in cross-validation (0.9738 for the zip model vs. 0.9682 for the ensemble) but scored worse on the public test data.

Final Choice

For the three main models that I tried, the AUROC scores on the public test set were the following: 0.97665 for the baseline, 0.97687 for the ensemble (gradient boosting + random forest + logistic regression), and 0.97574 for the zip model (gradient boosting + logistic regression). I went back and forth between the ensemble and the zip model for a long time since the ensemble scored better on the public test set but the zip model had a better cross-validation score. Ultimately, I went with the ensemble (Kaggle submission 2) because I used the public score to break the tie once the cross-validation scores were close. I decided the performance on the public test was more important since it is a direct part of our grade. In addition, the zip model was picked from seven candidates, so some of its edge in cross-validation might have been luck.

My main takeaway is that more isn't always better. Here, since we don't know our score on the private test set until after the deadline, it was important not to overfit. Adding extra features or trying more versions of the models to improve the public score can lead to the model overfitting to the noise of that specific data. On a similar note, more features in this case didn't help since last season's subscription alone separated a group where 82% bought from a group where under 1% did.

For the limitations, the cross-validation AUROC of 0.968 for my final ensemble may be a slightly optimistic estimate since it comes from the same data used to choose between candidates. The model also learns from the behavior of a single year, and we couldn't test it on a later period in the context of the competition. In addition, filling year columns with 0 to fix missing values was awkward for the logistic regression model. Lastly, I hand-picked the 17 features that I used and only tuned the baseline's settings and the logistic regression regularization. Based on these, the next steps I would continue with are 1. testing the concert files since I didn't use any of that data, 2. tuning the random forest and logistic regression and trying other boosting models, and 3. running more cross-validation repeats since the differences were so small.